Student Revision + Practice Sheet

---

1. What is LEFT JOIN?

`LEFT JOIN` returns:

- All records from the left table
- Matching records from the right table
- If no match is found in the right table, right table columns come as `NULL`

Simple meaning:

LEFT JOIN = Keep all rows from the left table and bring matching data from the right table

Example:

customers LEFT JOIN orders

This means:

Show all customers and their orders if available

If a customer has no order, the customer will still appear, but order columns will be `NULL`.

---

2. Difference Between INNER JOIN and LEFT JOIN

INNER JOIN

Returns only matching records from both tables.

Only customers who placed orders

LEFT JOIN

Returns all records from the left table and matching records from the right table.

All customers, even if they did not place orders

---

3. When Do We Use LEFT JOIN?

Use `LEFT JOIN` when unmatched records from the main table are also important.

Common use cases:

- Show all customers with orders if available
- Find customers who never ordered
- Find orders with invalid customer IDs
- Find payments with invalid order IDs
- Find products that were never sold
- Data quality checks
- Reconciliation reports
- Audit reports
- Reports where missing values should be shown as `NULL` or `0`

---

4. Complete SQL Setup

Use this setup from scratch.

DROP DATABASE IF EXISTS joins_practice;

```
CREATE DATABASE joins_practice;


USE joins_practice;
```

---

5. Create Tables

customers table

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50) NOT NULL,
    city VARCHAR(50),
    signup_date DATE
);
```

---

products table

```
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50)
);
```

---

orders table

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    order_amount DECIMAL(10,2),
    order_date DATE,
    order_status VARCHAR(30)
);
```

---

payments table

```
CREATE TABLE payments (
    payment_id INT PRIMARY KEY,
    order_id INT,
    payment_mode VARCHAR(30),
    payment_status VARCHAR(30),
    paid_amount DECIMAL(10,2),
    payment_date DATE
);
```

Note:

Foreign key constraints are not added here intentionally.
This helps us insert some invalid records and understand how `LEFT JOIN` helps in data quality checks.

---

6. Insert Sample Data

Insert customers

INSERT INTO customers

(customer_id, customer_name, city, signup_date)

VALUES

(1, 'Anurag', 'Bengaluru', '2026-01-10'),

(2, 'Stuti', 'Pune', '2026-02-15'),

(3, 'Rahul', 'Delhi', '2026-03-12'),

(4, 'Priya', 'Mumbai', '2026-04-05'),

(5, 'Neha', 'Bengaluru', '2026-05-20'),

(6, 'Aman', 'Hyderabad', '2026-06-10');

Insert products

INSERT INTO products

(product_id, product_name, category)

VALUES

(201, 'Laptop', 'Electronics'),

(202, 'Mouse', 'Accessories'),

(203, 'Keyboard', 'Accessories'),

(204, 'Monitor', 'Electronics'),

(205, 'Webcam', 'Accessories');

Insert orders

INSERT INTO orders

(order_id, customer_id, product_id, order_amount, order_date, order_status)

VALUES

(101, 1, 201, 50000.00, '2026-06-01', 'Delivered'),

(102, 1, 202, 1000.00, '2026-06-02', 'Delivered'),

(103, 2, 203, 2500.00, '2026-06-03', 'Delivered'),

(104, 99, 204, 15000.00, '2026-06-04', 'Delivered'),

(105, 3, 999, 3500.00, '2026-06-05', 'Delivered'),

(106, NULL, 201, 45000.00, '2026-06-06', 'Pending'),

(107, 5, 202, 1200.00, '2026-06-07', 'Cancelled'),

(108, 2, 201, 48000.00, '2026-06-08', 'Delivered');

Important observations:

- Anurag has 2 orders.
- Stuti has 2 orders.
- Rahul has 1 order with invalid product ID.
- Priya has no order.
- Aman has no order.
- Order 104 has invalid customer ID `99`.
- Order 105 has invalid product ID `999`.
- Order 106 has `customer_id = NULL`.

- Order 107 has no payment.

Insert payments

INSERT INTO payments

(payment_id, order_id, payment_mode, payment_status, paid_amount, payment_date)

VALUES

(1001, 101, 'UPI', 'Success', 50000.00, '2026-06-01'),

(1002, 102, 'Card', 'Success', 1000.00, '2026-06-02'),

(1003, 103, 'UPI', 'Failed', 0.00, '2026-06-03'),

(1004, 105, 'Wallet', 'Success', 3500.00, '2026-06-05'),

(1005, 999, 'UPI', 'Success', 3000.00, '2026-06-08'),

(1006, 108, 'Card', 'Success', 48000.00, '2026-06-08');

Important observations:

- Payment 1001 belongs to order 101.
- Payment 1002 belongs to order 102.
- Payment 1003 belongs to order 103, but payment failed.
- Payment 1004 belongs to order 105.
- Payment 1005 belongs to invalid order ID 999.
- Payment 1006 belongs to order 108.
- Order 107 has no payment.

---

7. Check the Data

SELECT * FROM customers;

SELECT * FROM products;

SELECT * FROM orders;

```
SELECT * FROM payments;
```

Check row counts:

```
SELECT COUNT(*) AS customer_count FROM customers;

SELECT COUNT(*) AS product_count FROM products;

SELECT COUNT(*) AS order_count FROM orders;

SELECT COUNT(*) AS payment_count FROM payments;
```

Expected counts:

customers = 6

products = 5

orders = 8

payments = 6

---

8. Basic LEFT JOIN Syntax

```
SELECT
    table1.column_name,
    table2.column_name
FROM table1
LEFT JOIN table2
    ON table1.common_column = table2.common_column;
```

Important:

LEFT JOIN always protects the left table.

So the table before `LEFT JOIN` is very important.

Level 1: Basic LEFT JOIN Questions

Question 1

Show all customers and their orders if available.

Solution

```
SELECT
    c.customer_id,
    c.customer_name,
    c.city,
    o.order_id,
    o.order_amount,
    o.order_status
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id;
```

Explanation

- All customers will come.
- Customers with orders will show order details.
- Customers without orders will show `NULL` in order columns.
- Priya and Aman will also come because they are in the left table.

Question 2

Show all customers with only customer name and order ID.

Solution

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id;
```

Explanation

This is a simple LEFT JOIN.

If a customer has multiple orders, the customer will appear multiple times.

Question 3

Show all products and their orders if available.

Solution

```
SELECT
    p.product_id,
```

```
    p.product_name,
    p.category,
    o.order_id,
    o.order_amount
FROM products p
LEFT JOIN orders o
    ON p.product_id = o.product_id;
```

Explanation

- All products will come.
- Product 205, Webcam, has no order.
- So order columns will be `NULL`.

Question 4

Show all orders and customer names if available.

Solution

```
SELECT
    o.order_id,
    o.customer_id,
    o.order_amount,
    c.customer_name
FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id;
```

Explanation

Here, orders is the left table.

So all orders will come.

Orders with invalid or missing customers will show `NULL` in customer columns.

Question 5

Show all payments and order details if available.

Solution

```
SELECT
    pay.payment_id,
    pay.order_id,
    pay.payment_mode,
    pay.payment_status,
    o.order_amount,
    o.order_status
FROM payments pay
LEFT JOIN orders o
    ON pay.order_id = o.order_id;
```

Explanation

Payment 1005 has order ID 999, but order 999 does not exist.

So order columns will be `NULL`.

Level 2: Missing Records and Data Quality Questions

Question 6

Find customers who never placed any order.

Solution

```
SELECT
    c.customer_id,
    c.customer_name,
    c.city
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Expected Result

Priya

Aman

Explanation

This is the most common LEFT JOIN pattern.

LEFT JOIN + WHERE right_table.key IS NULL

This finds records from the left table that do not have a match in the right table.

Question 7

Find orders with invalid customer IDs.

Solution

```
SELECT
    o.order_id,
    o.customer_id,
    o.order_amount,
    o.order_status
FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```

Expected Result

Order 104

Order 106

Explanation

- Order 104 has customer ID 99, which does not exist.
- Order 106 has NULL customer ID.

- Both do not have valid customer mapping.

Question 8

Find orders with invalid product IDs.

Solution

```
SELECT
    o.order_id,
    o.product_id,
    o.order_amount,
    o.order_status
FROM orders o
LEFT JOIN products p
    ON o.product_id = p.product_id
WHERE p.product_id IS NULL;
```

Expected Result

Order 105

Explanation

Order 105 has product ID 999.

Product 999 is not available in the products table.

Question 9

Find payments with invalid order IDs.

Solution

```
SELECT
    pay.payment_id,
    pay.order_id,
    pay.payment_mode,
    pay.payment_status,
    pay.paid_amount
FROM payments pay
LEFT JOIN orders o
    ON pay.order_id = o.order_id
WHERE o.order_id IS NULL;
```

Expected Result

Payment 1005

Explanation

Payment 1005 has order ID 999.

But order 999 does not exist in the orders table.

Question 10

Find orders where payment is not available.

Solution

```
SELECT
    o.order_id,
    o.customer_id,
    o.order_amount,
    o.order_status
FROM orders o
LEFT JOIN payments pay
    ON o.order_id = pay.order_id
WHERE pay.payment_id IS NULL;
```

Expected Result

Order 104

Order 106

Order 107

Explanation

These orders do not have any payment record.

This is useful in payment reconciliation.

Level 3: Reporting Questions

Question 11

Show every customer with total order amount.

If customer has no order, show total as 0.

Solution

```
SELECT
    c.customer_id,
    c.customer_name,
    COALESCE(SUM(o.order_amount), 0) AS total_order_amount
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY
    c.customer_id,
    c.customer_name;
```

Explanation

`COALESCE` is used to replace `NULL` with 0.

Priya and Aman will show total amount as 0.

Question 12

Show every customer with total number of orders.

Solution

```
SELECT
    c.customer_id,
    c.customer_name,
```

```
    COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY
    c.customer_id,
    c.customer_name;
```

Expected Logic

- Anurag = 2
- Stuti = 2
- Rahul = 1
- Neha = 1
- Priya = 0
- Aman = 0

Question 13

Show every customer with successful paid amount.

If no successful payment exists, show 0.

Solution

```
SELECT
    c.customer_id,
    c.customer_name,
    COALESCE(SUM(pay.paid_amount), 0) AS successful_paid_amount
FROM customers c
LEFT JOIN orders o
```

```
    ON c.customer_id = o.customer_id
LEFT JOIN payments pay
    ON o.order_id = pay.order_id
   AND pay.payment_status = 'Success'
GROUP BY
    c.customer_id,
    c.customer_name;
```

Explanation

The payment status filter is placed inside the `ON` condition.

This keeps all customers, even if they do not have successful payments.

Question 14

Show all products with total sales amount.

If product has no order, show 0.

Solution

```
SELECT
    p.product_id,
    p.product_name,
    p.category,
    COALESCE(SUM(o.order_amount), 0) AS total_sales
FROM products p
LEFT JOIN orders o
```

```
    ON p.product_id = o.product_id
GROUP BY
    p.product_id,
    p.product_name,
    p.category;
```

Explanation

Product 205, Webcam, has no order.

So it will show total sales as 0.

Question 15

Show all orders with payment status. If payment is missing, show `No Payment`.

Solution

```
SELECT
    o.order_id,
    o.order_amount,
    o.order_status,
    COALESCE(pay.payment_status, 'No Payment') AS payment_status
FROM orders o
LEFT JOIN payments pay
    ON o.order_id = pay.order_id;
```

Explanation

Orders without payment records will show `No Payment`.

Level 4: Common Mistakes and Tricky Concepts

Question 16

What is wrong with this query?

```
SELECT
    c.customer_name,
    COUNT(*) AS total_orders
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

Answer

This query uses `COUNT(*)`.

In LEFT JOIN, `COUNT(*)` counts the customer row also, even if the customer has no order.

So customers with no orders may get count 1.

Correct Solution

```
SELECT
    c.customer_name,
```

```
    COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

Rule

Use COUNT(right_table_column), not COUNT(*), when counting matched records in LEFT JOIN.

Question 17

Show all customers and only delivered orders if available.

Wrong Query

```
SELECT
    c.customer_name,
    o.order_id,
    o.order_status
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered';
```

Problem

The `WHERE` condition removes rows where order columns are `NULL`.

So customers with no orders will disappear.

This makes LEFT JOIN behave like INNER JOIN.

Correct Solution

```
SELECT
    c.customer_name,
    o.order_id,
    o.order_status
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
   AND o.order_status = 'Delivered';
```

Rule

In LEFT JOIN, if you want to preserve all left table rows, put right table filters inside ON.

Question 18

Show all customers and successful payment details if available.

Solution

```
SELECT
    c.customer_name,
    o.order_id,
```

```
    pay.payment_id,
    pay.payment_status,
    pay.paid_amount
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
LEFT JOIN payments pay
    ON o.order_id = pay.order_id
   AND pay.payment_status = 'Success';
```

Explanation

All customers will remain.

Only successful payment rows will match.

If payment is failed or missing, payment columns will come as `NULL`.

Question 19

Why does table order matter in LEFT JOIN?

Example 1

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
LEFT JOIN orders o
```

```
    ON c.customer_id = o.customer_id;
```

Meaning:

All customers + matching orders

Example 2

```
SELECT
    c.customer_name,
    o.order_id
FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id;
```

Meaning:

All orders + matching customers

Answer

Both queries are different because LEFT JOIN always preserves the left table.

Question 20

Can LEFT JOIN increase rows?

Answer

Yes.

LEFT JOIN can increase rows if the right table has multiple matches for one left table row.

Example:

Anurag has 2 orders.

So Anurag will appear 2 times when customers are left joined with orders.

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE c.customer_name = 'Anurag';
```

Expected:

Anurag - 101

Anurag - 102

Level 5: Multi-Table LEFT JOIN Questions

Question 21

Show all customers with order, product, and payment details wherever available.

Solution

```
SELECT
```

```
    c.customer_id,
    c.customer_name,
    o.order_id,
    p.product_name,
    o.order_amount,
    pay.payment_status
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
LEFT JOIN products p
    ON o.product_id = p.product_id
LEFT JOIN payments pay
    ON o.order_id = pay.order_id;
```

Explanation

All customers will come.

If order, product, or payment is missing, related columns will show `NULL`.

Question 22

Show all orders with customer, product, and payment details wherever available.

Solution

```
SELECT
    o.order_id,
```

```
    o.customer_id,
    c.customer_name,
    o.product_id,
    p.product_name,
    o.order_amount,
    pay.payment_status
FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id
LEFT JOIN products p
    ON o.product_id = p.product_id
LEFT JOIN payments pay
    ON o.order_id = pay.order_id;
```

Explanation

All orders will come because orders is the left table.

This is useful for order-level validation.

Question 23

Create an order quality report showing whether customer, product, and payment mapping exists.

Solution

```
SELECT
    o.order_id,
```

```
    o.customer_id,
    o.product_id,
    o.order_amount,

    CASE
        WHEN c.customer_id IS NULL THEN 'Invalid Customer'
        ELSE 'Valid Customer'
    END AS customer_check,

    CASE
        WHEN p.product_id IS NULL THEN 'Invalid Product'
        ELSE 'Valid Product'
    END AS product_check,

    CASE
        WHEN pay.payment_id IS NULL THEN 'No Payment'
        ELSE 'Payment Available'
    END AS payment_check

FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id
LEFT JOIN products p
    ON o.product_id = p.product_id
```

```
LEFT JOIN payments pay
    ON o.order_id = pay.order_id;
```

Explanation

This is a practical data engineering validation report.

It checks:

- Is customer valid?
- Is product valid?
- Is payment available?

Question 24

Find only problematic orders where customer, product, or payment is missing.

Solution

```
SELECT
    o.order_id,
    o.customer_id,
    o.product_id,
    o.order_amount,
    c.customer_name,
    p.product_name,
    pay.payment_id
FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id
```

```
LEFT JOIN products p
    ON o.product_id = p.product_id
LEFT JOIN payments pay
    ON o.order_id = pay.order_id
WHERE c.customer_id IS NULL
   OR p.product_id IS NULL
   OR pay.payment_id IS NULL;
```

Explanation

This query returns only records where at least one mapping is missing.

This is commonly used in ETL validation.

Level 6: Interview-Level Questions

Question 25

What is LEFT JOIN?

Answer

LEFT JOIN returns all records from the left table and matching records from the right table.

If there is no match in the right table, right table columns come as `NULL`.

Question 26

What is the difference between INNER JOIN and LEFT JOIN?

Answer

INNER JOIN returns only matching records from both tables.

LEFT JOIN returns all records from the left table and matching records from the right table.

If no match is found, right table columns are shown as `NULL`.

Question 27

When should we use LEFT JOIN?

Answer

Use LEFT JOIN when we want to keep all records from one table, even if matching records do not exist in another table.

Examples:

- All customers with orders if available
- Customers with zero orders
- Orders with invalid customers
- Products never sold
- Payments with invalid order IDs
- Data quality checks

Question 28

How do you find records present in table A but not in table B?

Answer

Use LEFT JOIN and filter where right table key is NULL.

Example:

Find customers who never ordered.

SELECT

```
    c.customer_id,
```

```
    c.customer_name
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Question 29

Why does LEFT JOIN sometimes behave like INNER JOIN?

Answer

This happens when we apply a filter on the right table in the `WHERE` clause.

Example:

```
WHERE o.order_status = 'Delivered'
```

This removes rows where right table columns are `NULL`.

So unmatched left table records disappear.

Correct approach:

```
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
   AND o.order_status = 'Delivered'
```

Question 30

What is the difference between filter in ON and filter in WHERE for LEFT JOIN?

Answer

Filter in `ON` controls which records from the right table should match.

Filter in `WHERE` filters the final result.

In LEFT JOIN, right table filters in `WHERE` can remove unmatched left table records.

Question 31

Can LEFT JOIN return more rows than the left table?

Answer

Yes.

If the right table has multiple matching records for one left table row, LEFT JOIN can return more rows.

Example:

One customer has 2 orders.

That customer will appear 2 times.

Question 32

Can LEFT JOIN return fewer rows than the left table?

Answer

Normally, LEFT JOIN returns at least all left table rows.

But if a `WHERE` condition filters the final output, then rows can become fewer.

Example:

WHERE o.order_status = 'Delivered'

This can remove customers with no orders.

Question 33

Why should we use COUNT(o.order_id) instead of COUNT(*) in LEFT JOIN?

Answer

`COUNT(*)` counts all rows, including unmatched left table rows.

So customers with no orders may show count 1.

`COUNT(o.order_id)` counts only matched order IDs.

So customers with no orders correctly show 0.

Question 34

What does NULL mean in LEFT JOIN output?

Answer

In most cases, NULL in right table columns means no matching record was found.

But NULL can also mean the actual value in the right table is NULL.

So we should understand the source data before assuming.

Question 35

Business asks: “Show all customers, even if they have not ordered.” Which join will you use?

Answer

Use LEFT JOIN with customers as the left table.

```
SELECT
    c.customer_id,
    c.customer_name,
    o.order_id
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id;
```

Question 36

Business asks: “Show only customers who have placed orders.” Which join will you use?

Answer

Use INNER JOIN.

```
SELECT
    c.customer_id,
    c.customer_name,
    o.order_id
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id;
```

Question 37

Business asks: “Find orders that do not have payments.” Which join will you use?

Answer

Use LEFT JOIN with orders as the left table.

```
SELECT
    o.order_id,
    o.order_amount
FROM orders o
LEFT JOIN payments pay
    ON o.order_id = pay.order_id
WHERE pay.payment_id IS NULL;
```

Question 38

Business asks: “Find payments where order does not exist.” Which join will you use?

Answer

Use LEFT JOIN with payments as the left table.

```
SELECT
    pay.payment_id,
    pay.order_id,
    pay.paid_amount
FROM payments pay
LEFT JOIN orders o
    ON pay.order_id = o.order_id
WHERE o.order_id IS NULL;
```

Question 39

Business asks: “Show all products including products with zero sales.” Which join will you use?

Answer

Use LEFT JOIN with products as the left table.

```
SELECT
    p.product_id,
    p.product_name,
    COALESCE(SUM(o.order_amount), 0) AS total_sales
FROM products p
LEFT JOIN orders o
    ON p.product_id = o.product_id
GROUP BY
    p.product_id,
    p.product_name;
```

Question 40

Business asks: “Create a data quality report for invalid customer, invalid product, and missing payment.” What will you do?

Answer

Use orders as the base table and LEFT JOIN with customers, products, and payments.

```
SELECT
    o.order_id,

    CASE
        WHEN c.customer_id IS NULL THEN 'Invalid Customer'
        ELSE 'Valid Customer'
    END AS customer_status,

    CASE
        WHEN p.product_id IS NULL THEN 'Invalid Product'
        ELSE 'Valid Product'
    END AS product_status,

    CASE
        WHEN pay.payment_id IS NULL THEN 'No Payment'
        ELSE 'Payment Available'
    END AS payment_status

FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id
LEFT JOIN products p
    ON o.product_id = p.product_id
LEFT JOIN payments pay
```

```
ON o.order_id = pay.order_id;
```

---

9. Final Revision Points

- LEFT JOIN keeps all records from the left table.
- Matching records from the right table are added.
- If no match is found, right table columns become `NULL`.
- Table order matters in LEFT JOIN.
- LEFT JOIN is useful for missing record checks.
- LEFT JOIN is heavily used in data quality validation.
- Use `LEFT JOIN + WHERE right_table.key IS NULL` to find unmatched records.
- Use `COALESCE` to replace NULL with 0 or meaningful text.
- Use `COUNT(right_table.column)`, not `COUNT(*)`, when counting matched records.
- Be careful with right table filters in the WHERE clause.
- Filters on the right table in WHERE can make LEFT JOIN behave like INNER JOIN.
- LEFT JOIN can return more rows than the left table if the right table has multiple matches.
